

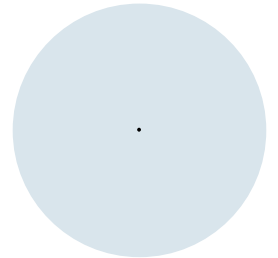
Einführung in die Programmierung (WIN+BIT)

Listen / Suchen und Maps. / Variable Parameteranzahl

Prof. Dr. Johannes C. Schneider / Prof. Dr. Markus Eiglsperger



Listen



Speichern von Objekten

- Eine Datei enthält alle Spieler der FIFA WM 2026 (WorldCup2026.csv)

No.	Country	Pos.	Player	Day	Month	Year	age	Club	
1	Algeria	GK	MASTIL	Melvin	19	February	2000	26 FC Stade Nyonnais (SUI)	
2	Algeria	DF	MANDI	Aissa	22	October	1991	34 Lille OSC (FRA)	
3	Algeria	DF	ABADA	Achraf	15	June	1999	26 USM Alger (ALG)	
4	Algeria	DF	TOUGAI	Mohamed	Amine	22	January	2000	26 Espérance De Tunisie (TUN)
5	Algeria	DF	BELAID	Zineddine	20	March	1999	27 JS Kabylie (ALG)	
6	Algeria	MF	ZERROUKI	Ramiz	26	May	1998	28 FC Twente (NED)	
7	Algeria	FW	MAHREZ	Riyad	21	February	1991	35 Al Ahli FC (KSA)	
8	Algeria	MF	AOUAR	Houssem	30	June	1998	27 Al Ittihad (KSA)	
9	Algeria	FW	GOUIRI	Amine	16	February	2000	26 Olympique Marseille (FRA)	

- Frage: Welche Spieler eines Vereins nehmen am Turnier teil?

Speichern von Objekten

- Wie würden gerne die Spieler speichern um Abfragen zu können:

```
private final Player[] players = new Player[2000];  
...
```

- Ein Spieler is definiert als:

```
public class Player {  
    private final String name;  
    private final String team;  
    private final String country;  
    ...  
}
```

Limitationen von Array I

- Arrays können benutzt werden um mehrere Instanzen eines gleichen Typs zu speichern:

```
// Speichert die Spieler aus der Datei in einem Array
private final Player[] players = new Player[1000];
// Anzahl der Spieler die aktuell im Array abgespeichert sind
// Soll ein neuer Spieler hinzugefügt werden, dann an die Stelle count
private int countPlayer = 0;
```

Probleme:

- Genaue Anzahl muss vorher bekannt sein!
- Wenn dies nicht möglich ist:
 - Array erzeugen, der auf jeden Fall groß genug ist
 - Unbenutzte Felder leer lassen (null)

Limitationen von Array II

- Neue Elemente werden hinten angehängt:

```
private void addPlayer(Player player) {  
    players[countPlayer] = player;  
    countPlayer++;  
}
```

Probleme:

- Unnötiger Speicherplatz wird reserviert
- In Extremfällen kann Platz nicht ausreichen
- Umständliche Buchhaltung mit `countPlayer`
- Fehleranfällig (Ausnahmebehandlung für `null`)

Limitationen von Array III

– Suche

```
public Player[] searchVerein(String searchString) {  
    Player[] result = new Player[1000];  
    int count = 0;  
    for (int i = 0; i < countPlayer; i++) {  
        if (players[i].getTeam().contains(searchString)) {  
            result[count++] = players[i];  
        }  
    }  
    return result;  
}
```

Probleme:

- Unnötiger Speicherplatz wird reserviert
- In Extremfällen kann Platz nicht ausreichen
- Umständliche Buchhaltung mit `countPlayer`
- Fehleranfällig (Ausnahmebehandlung für `null`)

Limitationen von Array III

– Suche

```
public Player[] searchVerein(String searchString) {  
    Player[] result = new Player[1000];  
    int count = 0;  
    for (Player p : players) {  
        if (p != null && p.getTeam().contains(searchString)) {  
            result[count++] = p;  
        }  
    }  
    return result;  
}
```

Probleme:

- Unnötiger Speicherplatz wird reserviert
- In Extremfällen kann Platz nicht ausreichen
- Umständliche Buchhaltung mit `countPlayer`
- Fehleranfällig (Ausnahmebehandlung für `null`)

ArrayList – Deklaration und Erzeugung

- ArrayLists sind eine Alternative um mehrere Instanzen eines gleichen Typs zu speichern:

```
private final ArrayList<Player> players = new ArrayList<>();
```

Alternative Deklaration:



Typ-Parameter

```
private final ArrayList<Player> players = new ArrayList<Player>();
```

ArrayList benötigen als Parameter den Typ, der in der Liste verwaltet wird.

Vorteile:

- Genaue Anzahl muss vorher **nicht** bekannt sein!

Nachteile:

- Sind etwas aufwändiger implementiert als Arrays

ArrayLists – Elemente hinzufügen

- Neue Elemente werden hinten angehängt:

```
private void addPlayer(Player player) {  
    players.add(player);  
}
```

Vorteile:

- **Kein** unnötiger Speicherplatz wird reserviert
- **Kein** unnatürliches Limit für die Größe
- **Keine** umständliche Buchhaltung mit `countPlayer`
- **Weniger** fehleranfällig (Keine ausnahmebehandlung für `null`)

ArrayLists - Iteration

- Neue Elemente werden hinten angehängt:

```
public Player[] searchVerein(String searchString) {  
    Player[] result = new Player[1000];  
    int count = 0;  
    for (Player p : players) {  
        if (p.getTeam().contains(searchString)) {  
            result[count++] = p;  
        }  
    }  
    return result;  
}
```

Vorteile:

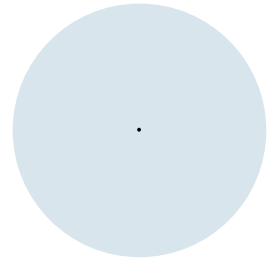
- **Kein** unnötiger Speicherplatz wird reserviert
- **Kein** unnatürliches Limit für die Größe
- **Keine** umständliche Buchhaltung mit `countPlayer`
- **Weniger** fehleranfällig (Keine ausnahmebehandlung für `null`)

Zusammenfassung: ArrayList

ArrayList<E>

- `new ArrayList<E>()` erzeugt neue ArrayList mit Werte-Typ E
- `boolean add(E e)` fügt Element am Ende hinzu
- `E get(int index)` gibt Element an Position `index` zurück
- `boolean contains(E e)` prüft, ob Element `e` enthalten ist
- `E remove(E e)` entfernt Eintrag `e`
- `int size()` Anzahl der Einträge

Wrapper-Klassen



Primitive Datentypen als Typ-Parameter?

- Die primitiven Datentypen int, long, double, float, char, boolean, etc. lassen sich **nicht** als Typparameter verwenden:

```
ArrayList<int> list = new ArrayList<int>();
```

Error: Type argument cannot be of primitive type.

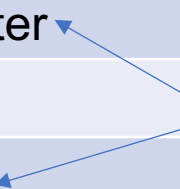
- Java erwartet hier Referenzdatentypen!
- Als Lösung bietet Java zu den primitiven Datentypen passende Referenzdatentypen an: Long, Double, Integer, ...

Wrapper-Klassen für primitive Datentypen

Java bietet die folgenden Wrapper-Klassen an:

Primitiver Typ	Wrapper-Klasse
boolean	Boolean
byte	Byte
char	Character
float	Float
int	Integer
long	Long
short	Short
double	Double

Einfach Großbuchstabe am Anfang,
außer bei diesen beiden.



Umwandlung (1)

Umwandlung primitiver Typ in Wrapper-Typ:

```
int primitive = 5;  
Integer wrapped = Integer.valueOf(primitive);
```

Der Compiler macht das allerdings auch automatisch. Daher kann man einfach folgendes schreiben:

```
int primitive = 5;  
Integer wrapped = primitive;
```

oder direkt:

```
Integer wrapped = 5;
```

Man nennt dies "**Autoboxing**" (int wird in Integer "verpackt")

jshell>

Umwandlung (2)

Umwandlung Wrapper-Typ in primitiven Typ:

Das funktioniert genauso für andere Wrapper-Typen, mit z.B. `.doubleValue()`, `.booleanValue()`, ...

```
Integer wrapped = Integer.valueOf(42);  
int primitive = wrapped.intValue();
```

Der Compiler macht das allerdings auch automatisch. Daher kann man einfach folgendes schreiben:

```
Integer wrapped = Integer.valueOf(42);  
int primitive = wrapped;
```

Man nennt dies "**Autounboxing**" (der int-Wert wird aus der Wrapper-Klasse "ausgepackt")

js11>

Weitere Beispiele

<https://docs.oracle.com/javase/tutorial/java/data/autoboxing.html>

Auto(un)boxing mit Generics

Im Zusammenhang mit Generics ist das Auto(un)boxing sehr praktisch:

1. Man kann einerseits primitive Werte über die Wrapper-Typen speichern
2. und andererseits mit ihnen wie mit primitiven Werten rechnen

1. Boxing (Umwandlung
primitiver Typ in Wrapper-Typ)

```
KeyValue<String, Integer> nameToAge = new KeyValue<String, Integer>("Chris", 23);  
int birthYear = 2022 - nameToAge.value;
```

2. Unboxing (Umwandlung
Wrapper-Typ in primitivem Typ)

Vorsicht mit null

Achtung: Die Wrapper-Klassen sind Referenz-Typen, können also auch den Wert null haben. Auto-Unboxing von null schlägt fehl:

```
Integer boxedInt = null;  
int primitiveInt = 5;  
int sum = boxedInt + primitiveInt;
```

NullPointerException

Warum? Der Compiler erzeugt aus obigem Code das hier:

```
Integer boxedInt = null;  
int primitiveInt = 5;  
int sum = boxedInt.intValue() + primitiveInt;
```

`null.intValue()` kann nicht aufgerufen werden!

jsHELL>

Wrapper-Typen: Empfehlung

- Wrapper-Typen sind zwingend notwendig, um primitive Datentypen bei Typ-Parametern zu verwenden

```
ArrayList<int> list = new ArrayList<int>;
```

funktioniert nicht (Compiler-Fehler)



```
ArrayList<Integer> list = new ArrayList<Integer>;
```

- Im Normalfall sollte man jedoch die primitiven Datentypen bevorzugen, z.B. als Attribute in Klassen oder als Variablen, um versehentliche NullPointerExceptions zu vermeiden

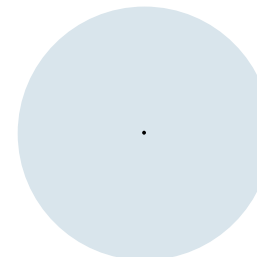
```
for (Integer i = 0; i < 10; i++) { ... }
```

funktioniert zwar, aber unnötige Umwandlung Integer ↔ int



```
for (int i = 0; i < 10; i++) { ... }
```

Suchen und Maps



Suchen

- Ausgangslage: Eine Menge von Objekten.
- Fragen?
 - Enthält die Menge ein bestimmtes Objekt?
 - Wie oft enthält es das Objekt?

Bsp: Pkw Maut mit Kennzeichen

- Ausgangslage:
 - Menge aller Kennzeichen **M**, die für diesem Tag die Pkw-Maut bezahlt haben.
- Frage:
 - Hat ein Auto mit Kennzeichen **K** die Maut für diesen Tag bezahlt?

Die Kennzeichen, für die die Maut bezahlt wurde, sind in einer Datei `kennzeichen.txt` abgespeichert.

Vorgehen:

- Lese Datei `kennzeichen.txt` ein und speichere die Kennzeichent in einer Liste.
- Frage den Benutzer nach einem Kennzeichen
- Überprüfe, ob das Kenzeichen in der Liste vorhanden ist, indem jedes Element verglichen wird.
- Wiederhole bis der Benutzer “exit” tippt.

Bsp: Pkw Maut mit Kennzeichen - Liste

Vorgehen:

- Lese Datei `kennzeichen.txt` ein und speichere die Kennzeichen in einer Liste.
-

```
Scanner scanner = new Scanner(mautFile).useDelimiter("\n");
ArrayList<String> registrierteKennzeichen = new ArrayList<>();
while (scanner.hasNextLine()) {
    String line = scanner.nextLine();
    registrierteKennzeichen.add(line);
}
```

Bsp: Pkw Maut mit Kennzeichen - Liste

Vorgehen:

- ...
- Überprüfe, ob das Kennzeichen in der Liste vorhanden ist, indem jedes Element verglichen wird.
- ...

```
boolean found = false;
for (String k : registrierteKennzeichen) {
    if (k.equals(eingabe)) {
        found = true;
    }
}
if (found) {
    System.out.println("Maut bezahlt");
} else {
    System.out.println("ACHTUNG: Keine Maut bezahlt: " + eingabe);
}
```

Bsp: Pkw Maut mit Kennzeichen - Liste

- Ausgangslage:
 - Menge aller Kennzeichen **M**, die für diesem Tag die Pkw-Maut bezahlt haben.
- Frage:
 - Welche Fahrzeuge in einer Liste **N** haben die Maut für diesen Tag **nicht** bezahlt?

Die Kennzeichen, für die die Maut bezahlt wurde, sind in einer Datei `kennzeichen.txt` abgespeichert.

Die Kennzeichen, die überprüft werden sollen, sind in einer Datei `check.txt` abgespeichert.

Vorgehen:

- Lese Datei `kennzeichen.txt` ein und speichere die Kennzeichen in einer **Liste**.
- Lese die Datei `check.txt` Zeile für Zeile ein
 - Überprüfe, ob das Kenzeichen in der Liste vorhanden ist
- Gebe das Ergebnis aus

Bsp: Pkw Maut mit Kennzeichen - Liste

Vorgehen:

- Lese Datei `kennzeichen.txt` ein und speichere die Kennzeichen in einer **Liste**.
- Lese die Datei `check.txt` Zeile für Zeile ein
 - Überprüfe, ob das Kennzeichen in der Liste vorhanden ist, wenn nein erhöhe Zähler um 1
- Gebe das Ergebnis aus

```
while (scannerCheckFile.hasNextLine()) {  
    String line = scannerCheckFile.nextLine();  
    boolean found = registrierteKennzeichen.contains(line);  
    if (!found) {  
        System.out.println("Keine Maut bezahlt: "+line);  
    }  
}
```

Bsp: Pkw Maut mit Kennzeichen - Map

- Ausgangslage:
 - Menge aller Kennzeichen **M**, die für diesem Tag die Pkw-Maut bezahlt haben.
- Frage:
 - Welche Fahrzeuge in einer Liste **N** haben die Maut für diesen Tag **nicht** bezahlt?

Die Kennzeichen, für die die Maut bezahlt wurde, sind in einer Datei `kennzeichen.txt` abgespeichert.

Die Kennzeichen, die überprüft werden sollen, sind in einer Datei `check.txt` abgespeichert.

Vorgehen:

- Lese Datei `kennzeichen.txt` ein und speichere die Kennzeichen in einer **Map**.
- Lese die Datei `check.txt` Zeile für Zeile ein
 - Überprüfe, ob das Kennzeichen in der **Map** vorhanden ist
- Gebe das Ergebnis aus

Bsp: Pkw Maut mit Kennzeichen - Map

Vorgehen:

- Lese Datei kennzeichen.txt ein und speichere die Kennzeichen in einer Map.
-

```
Scanner scannerMautFile = new Scanner(mautFile).useDelimiter("\n");
HashMap<String,String> registrierteKennzeichen = new HashMap<>();
while (scannerMautFile.hasNextLine()) {
    String line = scannerMautFile.nextLine();
    registrierteKennzeichen.put(line,"bezahlt");
}
```

Bsp: Pkw Maut mit Kennzeichen - Map

Vorgehen:

- ...
- Überprüfe, ob das Kenzeichen in der Liste vorhanden ist, indem jedes Element verglichen wird.
- ...

```
while (scannerCheckFile.hasNextLine()) {  
    String line = scannerCheckFile.nextLine();  
    if (!registrierteKennzeichen.containsKey(line)) {  
        System.out.println("Keine Maut bezahlt: "+line);  
    }  
}
```

Bsp: Pkw Maut mit Kennzeichen - Count

- Ausgangslage:
 - Menge aller Kennzeichen **M**, die für diesem Tag die Pkw-Maut bezahlt haben.
- Frage:
 - Welche Fahrzeuge in einer Liste **N** haben die Maut für diesen Tag **nicht** bezahlt?
 - **Wie oft wurde ein Fahrzeug, dass die Maut bezahlt hat, an diesem Tag registriert?**

Vorgehen:

- Lese Datei `kennzeichen.txt` ein und speichere die Kennzeichen in einer **Map**
 - Speichere einen Zähler 0 für jedes Kennzeichen
- Lese die Datei `check2.txt` Zeile für Zeile ein
 - Überprüfe, ob das Kennzeichen in der **Map** vorhanden ist
 - Wenn Nein -> Ausgabe, dass Fahrzeug keine Maut bezahlt hat
 - Wenn Ja -> Erhöhe Zähler
- Gebe das Ergebnis aus

Bsp: Pkw Maut mit Kennzeichen - Count

Vorgehen:

- Lese Datei kennzeichen.txt ein und speichere die Kennzeichen in einer Map.
-

```
Scanner scannerMautFile = new Scanner(mautFile).useDelimiter("\n");
HashMap<String,Integer> registrierteKennzeichen = new HashMap<>();
while (scannerMautFile.hasNextLine()) {
    String line = scannerMautFile.nextLine();
    registrierteKennzeichen.put(line,0);
}
```

Bsp: Pkw Maut mit Kennzeichen - Count

Vorgehen:

- Lese die Datei `check2.txt` Zeile für Zeile ein
 - Überprüfe, ob das Kenzeichen in der **Map** vorhanden ist
 - Wenn Nein -> Ausgabe, dass Fahrzeug keine Maut bezahlt hat
 - Wenn Ja -> Erhöhe Zähler
- Überprüfe, ob das Kenzeichen in der Liste vorhanden ist, indem jedes Element verglichen wird.

```
while (scannerCheckFile.hasNextLine()) {  
    String line = scannerCheckFile.nextLine();  
    Integer count = registrierteKennzeichen.get(line);  
    if (count != null) {  
        registrierteKennzeichen.put(line, count+1);  
    } else {  
        System.out.println("Keine Maut bezahlt: "+line);  
    }  
}
```

Zusammenfassung: HashMap

- Was ist eine HashMap?
- Datenstruktur zur **Speicherung von Schlüssel-Wert-Paaren**
- Teil des java.util-Frameworks
- Jeder **Key ist eindeutig**, Werte dürfen mehrfach vorkommen

Typ-Parameter: Schlüssel

Typ-Parameter: Wert

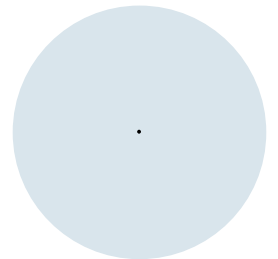
```
HashMap<String,Integer> registrierteKennzeichen = new HashMap<>();  
while (scannerMautFile.hasNextLine()) {  
    String line = scannerMautFile.nextLine();  
    registrierteKennzeichen.put(line,0);  
}
```

Zusammenfassung: Map

HashMap<K, V>

- `new HashMap<K, V>()` erzeugt neue HashMap mit Schlüssel-Typ `K` und Werte-Typ `V`
- `V put(K key, V value)` fügt Eintrag hinzu / überschreibt Wert
- `V get(K key)` gibt Wert zum Schlüssel zurück, `null` wenn Schlüssel nicht existiert
- `V containsKey(K key)` prüft, ob Key existiert
- `boolean remove(K key)` entfernt Eintrag
- `int size()` Anzahl der Einträge
- `Set<K> keySet()` alle Schlüssel

Variable Parameterzahl



Motivation

- Wie viele Parameter hat die Funktion `System.out.println`?

```
System.out.println("Hallo Welt!");
```

→ einen Parameter (String "Hallo Welt!").

Motivation

- Wie viele Parameter hat die Funktion `System.out.printf`?

```
System.out.printf("Hallo Welt!%n");
```

→ einen Parameter?

Erinnerung: %n erzeugt Zeilenumbruch

```
System.out.printf("Hallo %s!%n", "Welt");
```

→ Zwei Parameter?

```
System.out.printf("Hallo %s! Die Antwort ist %d.%n", "Welt", 42);
```

→ Drei Parameter?

Motivation

Wie viele Parameter hat die Funktion `System.out.printf`?

→ Bis zu n Parameter mit $n = 5?$ $10?$ $100?$

→ Oder sogar beliebig viele Parameter?

... Auflösung folgt am Ende 😊

Variable Parameterzahl

Zunächst einfacheres Beispiel

- Gesucht: Methode, mit der 2, 3, 4, ... Zahlen summiert werden können

Ansatz 1: Überladen

```
public static int sumUp(int a, int b) {  
    return a + b;  
}  
  
public static int sumUp(int a, int b, int c) {  
    return a + b + c;  
}  
  
public static int sumUp(int a, int b, int c, int d) {  
    return a + b + c + d;  
}  
  
...
```



code16.
SumItUp1

Ansatz 2: Übergabe als Array

```
public static int sumUp(int[] values) {  
    int sum = 0;  
    for (int i = 0; i < values.length; i++) {  
        sum = sum + values[i];  
    }  
    return sum;  
}
```

- Nachteil: Aufrufer muss Werte vor Aufruf der Methode in Array verpacken

```
int[] myValues = {23, 42, 1414, 1418, 120};  
int sum = sumUp(myValues);  
  
// oder kürzer  
int sum = sumUp(new int[] {23, 42, 1414, 1418, 120});
```



Erinnerung: for-each-Schleife verwenden!

```
public static int sumUp(int[] values) {  
    int sum = 0;  
    for (int i = 0; i < values.length; i++) {  
        sum = sum + values[i];  
    }  
    return sum;  
}
```

```
public static int sumUp(int[] values) {  
    int sum = 0;  
    for (int a : values) {  
        sum = sum + a;  
    }  
    return sum;  
}
```

So ist es eleganter!

Ansatz 3: Variable Parameterzahl

```
public static int sumUp(int... values) {  
    int sum = 0;  
    for (int a : values) {  
        sum = sum + a;  
    }  
    return sum;  
}  
  
int sum = sumUp(23, 42, 1414, 1418, 120);
```

- Vorteil: Aufrufer kann beliebige Anzahl Parameter des gleichen Typs angeben (0, 1, 2, 3, ...)
- Parameter werden automatisch in Array verpackt



Variable Parameterzahl, Besonderheiten

- Wenn man mindestens x Parameter erzwingen möchte, muss man diese explizit angeben.
Beispiel für mindestens zwei Parameter:

```
public static int sumUp(int a, int b, int... values) { ... }
```

- Variable Parameteranzahl nur für letzten Parameter möglich
 - Compiler muss unterscheiden können, welche Parameter automatisch in ein Array verpackt werden

```
public static int sumUp(int... values, int a) { ... }
```

- Man kann Methodennamen mit Methode für spezielle Parameterzahl überladen – diese wird dann bevorzugt aufgerufen

```
public static int sumUp(int a, int b, int... values) { ... }  
public static int sumUp(int a, int b, int c) { ... }
```



code16.
VarArgsAd-
vancedMain

Variable Parameterzahl, Besonderheiten

- Man kann auch ein Array als Parameter übergeben

```
public static int sumUp(int a, int b, int... values) { ... }  
  
sumUp(1, 2, new int[]{3, 4});
```

- Dadurch auch Wert `null`, möglich. Diesen Fall muss man abfangen!

```
sumUp(1, 2, null);
```



code16.

VarArgsAd-
vancedMain

printf

- Zur Ausgangsfrage: Wie viele Parameter hat die Funktion `System.out.printf`?

printf

```
public PrintStream printf(String format, Object... args)
```

→ beliebig viele!

Variable Parameterzahl

- Beispiele aus dem SDK:
 - `System.out.printf()/.format()` – formatierte Ausgabe von Werten
 - `java.lang.Class.getMethod()` – liefert Methode einer Klasse
 - `java.lang.reflect.Method.invoke()` – ruft Methode einer Klasse auf
 - `java.lang.ProcessBuilder()` – dient zum Starten von weiteren Prozessen